



CHALLENGE 14 / 20

THE TIMESTAMPS THAT LIED

TEMPORAL INTEGRITY AND CAUSAL ORDERING

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

A telecom operator with 90 million subscribers was breached across its core network management and billing systems: 2,300 servers, 140 network elements and six data centres. Evidence contradicts itself in time. Files are modified before they are created. Sessions end before they start. Time-synchronisation logs show four servers stepped by days. One hypervisor's clock was set deliberately.

Some devices write local time with no zone, others use GPS time with a different leap-second count. A log forwarder replaced event times with ingest times, and the central search platform was re-indexed using ingestion time. The attacker changed clocks to break correlation.

A regulator and a prosecutor need a timeline, and a defence expert will attack every ordering claim.

2 | PROBLEM STATEMENT

Develop a **Temporal Integrity and Causal Ordering System** that detects drift, steps, manipulation and time-standard errors across heterogeneous sources, recovers ordering constraints that do not depend on recorded wall-clock time, and derives bounded time windows and a partial order of events.

The system must state exactly which ordering claims would fail if any given clock or source were proven false.

Hard constraints

- The attacker manipulated clocks specifically to defeat correlation, and which clocks is unknown.
- Time zones, time standards, leap seconds and ingestion rewriting must all be handled.
- The system must work at hundreds of millions of events.
- Each ordering claim must come with a checkable justification.
- The result must withstand adversarial challenge, which means robustness to the loss of any single source.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Clock Fault Detection	Ability to detect drift, steps and deliberate manipulation.
02	Time-Standard Normalisation	Ability to reconcile zones, time standards and leap seconds.
03	Ingestion-Rewrite Recovery	Ability to recover original event times after forwarding or re-indexing.
04	Clock-Independent Ordering	Ability to derive ordering from relationships that do not use wall-clock time.
05	Bounded Time Windows	Ability to give event times as bounded windows.
06	Source-Loss Robustness	Ability to show which claims survive the loss of any source.
07	Machine-Checkable Justification	Ability to justify each ordering claim in a form that can be checked.
08	Scale Handling	Ability to process hundreds of millions of events.

The system must clearly distinguish between:

CLOCK-VERIFIED	DRIFTED	MANIPULATED	ORDER-ONLY	UNKNOWN
----------------	---------	-------------	------------	---------

A recorded timestamp must not be treated as true time without evidence.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated multi-site environment with several clock faults
02	Deliberate manipulation by an adversary
03	Detection of each clock fault
04	Ordering without trusting recorded time
05	Time windows with bounds
06	Robustness analysis to the loss of a source
07	Checkable justification of an ordering claim

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.